

Quantum Processes and Computation Mini-Project

1633814

Michaelmas Term 2023

Question 1

The labelled ZX -rules shown below [6] for $\alpha, \beta \in [0, 2\pi)$ and $a \in \{0, 1\}$ are as taught in lectures [2], and will be used to justify the equations below.

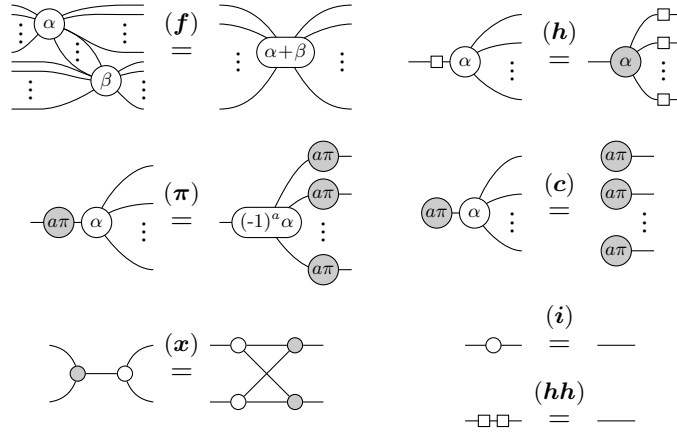
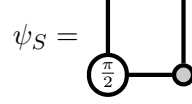


Figure 1: ZX -rules presented in [6]

(a) Let

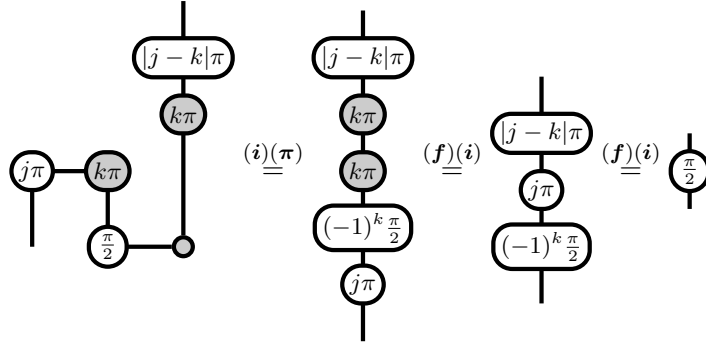


Note for $j, k \in \{0, 1\}$,

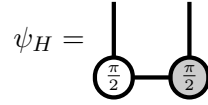
$$|j - k|\pi + j\pi + (-1)^k \frac{\pi}{2} = 2n\pi + \frac{\pi}{2}$$

for some $n \in \{0, 1\}$.

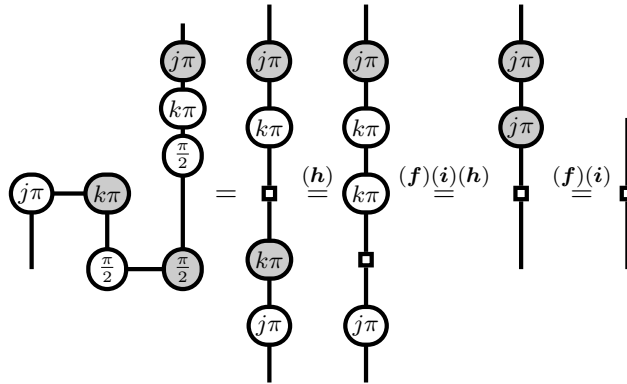
Then, using ψ_S , a 2-qubit measurement, and classically controlled Pauli X/Z gates, the S -gate is obtained as desired.



(b) Let



Then, also using the S -gate from part (a),



$\psi_{CNOT} =$ 

so we have, using 2 two-qubit measurements, and classically controlled Pauli X/Z gates,

$$\psi_{CNOT} =$$

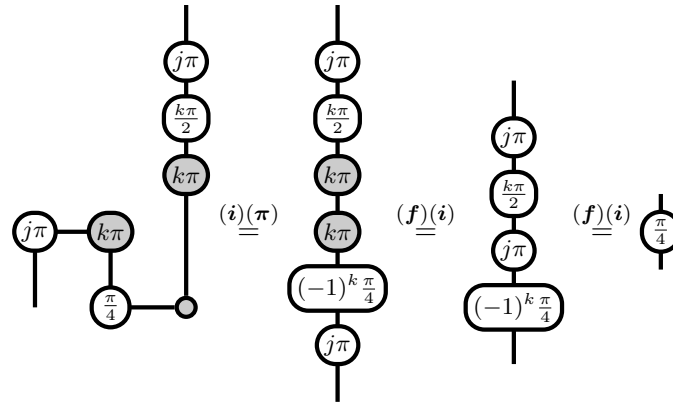
(d) No, since the Bell model only has $\circ \frac{n\pi}{2}$ -phase gates for some $n \in \{0, 1, 2, 3\}$, $\circ \frac{\pi}{4}$ -phase gates cannot be constructed.

Definition 1.1 ([2] **Def 9.120.**). [Graph state] A graph state is a state whose ZX -diagram consists only of (0-phase) $\circ$ -spiders and H -gates where:

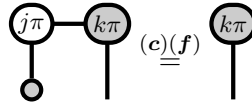
- Definition 1.2** ([2] Def 9.103.). [Clifford diagram] A Clifford diagram is a ZX-diagram where the phases are restricted to integer multiples of $\frac{\pi}{2}$.

Definition 1.4 ([2]). [Local Clifford unitaries] Local Clifford unitaries are unitaries expressible as parallel compositions of single-system Clifford diagrams.

have



(e) It is just as powerful as before. Simply use a two-qubit measurement on the Z -basis state as follows to obtain the equivalent Z -basis measurement. Any diagram which requires the single-qubit measurement can then be replaced. The classical value of the first qubit (j) is essentially discarded.



Question 2

Background

In the Deutsch-Josza algorithm, the goal is to differentiate between two classes of functions, using a single quantum query [3] - namely, whether the function is *constant*, or *balanced*.

What if, instead of two classes of functions, there were instead, 2^n possible functions to differentiate between? More concretely, what if there were an n -bit string encoded in the oracle function we had access to, and we need to obtain the string using our quantum query?

Mathematically, the problem is as follows [1].

Given. A function

$$f_a : \{0, 1\}^n \rightarrow \{0, 1\}$$

with the promise that, for some n -bit string $a \in \{0, 1\}^n$,

$$f_a(x) = a \cdot x \equiv \left(\sum_{i=1}^n a_i x_i \right) \pmod{2},$$

where a_i, x_i are the i -th bits of a, x respectively.

Problem. What is the exact string a ?

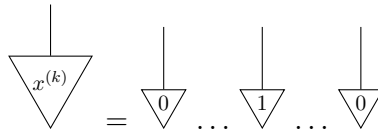
Note the use of the $\cdot$ operator, which will continue to be used in the rest of this paper.

In a way, the problem above can also be seen as a search algorithm, where we are searching for the exact indices k where the value of bit a_k is 1.

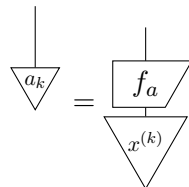
In [8], the problem is described as a coin weighing problem, where the set of defective coins need to be found in the total set of coins. Here, a spring-scale is used and gives the Hamming weight instead, which is the total number of 1 bits in the the bitwise product of a and x , $w_h(a \wedge x)$. However, this paper will only make use of the parity of the bitwise product, $a \cdot x$.

Classical Solution

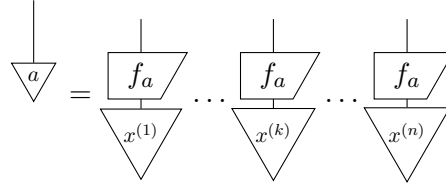
The classical solution to this problem would simply to figure out the string a bit by bit. Each bit a_k can be determined by using the query string $x^{(k)}$ of all 0's, except for bit $x_k = 1$.



Each query $x^{(k)}$ will correspond to bit a_k



so

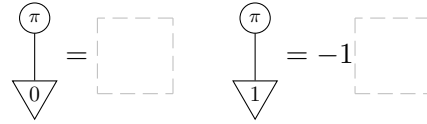


With n queries, it is clear the classical solution requires $O(n)$ queries.

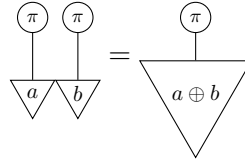
Quantum Analysis

Owing to the problem's similarity to both the problem used in Deutsch-Josza [3], as well as Grover's search algorithm [5], it is reasonable to start by analysing the methods used in those algorithms.

The following fact is essential to both algorithms, as seen in **Section 12.2** of [2].

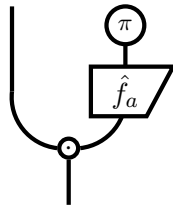


Corollary 1.10.

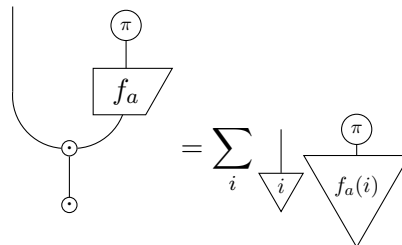


for $a, b \in \{0, 1\}$, where $\oplus$ is the XOR operator.

We set the second input of the oracle to once again be the $\circ$ -state with a π , to make use of the above property.



Like in [2], if f_a is applied to "all inputs together", then it results in the following, where we leave the number for later use.



It can be seen that measuring at this point does not help as much, since it is still a sum of different states that are not our target string a . We then have to find a unitary which maps the state to something more useful, which does not seem straightforward. So, we start by proving the following lemma, using the Hadamard gate.

Lemma 1.11. Applying the n -bit Hadamard gate (denoted $H^{(n)}$) in parallel to our n -bit input gives the following.

$$\boxed{H^{(n)}} = \frac{1}{\sqrt{2^n}} \sum_{j \in \{0,1\}^n} \begin{array}{c} \pi \\ \downarrow \\ \triangleleft i \cdot j \end{array} \downarrow j$$

Proof. First, we see the Hadamard applied on a single bit i_k at index k .

$$\boxed{H} = \begin{array}{c} \downarrow \\ \triangleleft i_k \end{array} = \frac{1}{\sqrt{2}} \left(\begin{array}{c} \downarrow \\ \triangleleft 0 \end{array} + (-1)^{i_k} \begin{array}{c} \downarrow \\ \triangleleft 1 \end{array} \right)$$

Then, we note that the 1 and -1 can be rewritten.

$$\begin{aligned} \begin{array}{c} \square \end{array} &= \begin{array}{c} \pi \\ \downarrow \\ \triangleleft 0 \end{array} = \begin{array}{c} \pi \\ \downarrow \\ \triangleleft i_k \cdot 0 \end{array} \\ (-1)^{i_k \cdot 1} &= \begin{array}{c} \pi \\ \downarrow \\ \triangleleft i_k \cdot j_k \end{array} = \begin{array}{c} \pi \\ \downarrow \\ \triangleleft i_k \cdot 1 \end{array} \end{aligned}$$

Then, we have for all bits i_k ,

$$\boxed{H} = \sum_{j_k \in \{0,1\}} \begin{array}{c} \pi \\ \downarrow \\ \triangleleft i_k \cdot j_k \end{array} \downarrow j_k$$

Since we can see that

$$\begin{array}{c} \pi \\ \downarrow \\ \triangleleft i \cdot j \end{array} = \begin{array}{c} \pi \\ \downarrow \\ \triangleleft i_1 \cdot j_1 \end{array} \dots \begin{array}{c} \pi \\ \downarrow \\ \triangleleft i_n \cdot j_n \end{array}$$

Then

$$\boxed{H^{(n)}} = \boxed{H} \dots \boxed{H} = \frac{1}{\sqrt{2^n}} \sum_{j \in \{0,1\}^n} \begin{array}{c} \pi \\ \downarrow \\ \triangleleft i \cdot j \end{array} \downarrow j$$

□

$$\begin{aligned}
& \frac{1}{\sqrt{2^n}} \text{ (Diagram: A box labeled } H^{(n)} \text{ connected to a circle with } \pi \text{, which is connected to a box labeled } f \text{, which is connected to a circle with } \pi \text{.)} \\
&= \frac{1}{\sqrt{2^n}} \sum_{i \in \{0,1\}^n} \text{ (Diagram: A box labeled } H^{(n)} \text{ connected to a circle with } \pi \text{, which is connected to a box labeled } f_a(i) \text{.)} \\
&\stackrel{1.11}{=} \frac{1}{2^n} \sum_{i,j \in \{0,1\}^n} \text{ (Diagram: A box labeled } H^{(n)} \text{ connected to a circle with } \pi \text{, which is connected to a box labeled } f_a(i) \text{.)} \\
&\stackrel{1.10}{=} \frac{1}{2^n} \sum_{i,j \in \{0,1\}^n} \text{ (Diagram: A box labeled } H^{(n)} \text{ connected to a circle with } \pi \text{, which is connected to a box labeled } f_a(i) \text{.)}
\end{aligned}$$

Lemma 1.12.

Proof. First, we simplify the expression

where the $\oplus$ between n -bit strings a, j is simply the bitwise XOR resulting in an n -bit string. This is true since for each bit index k ,

where if $i_k = 0$, both sides are 0, and if $i_k = 1$, both sides are equal to $a_k \oplus j_k$.

$$\begin{array}{c} \downarrow \\ \triangle j \end{array} = \begin{array}{c} \downarrow \\ \triangle a \end{array}$$

then $i_k \cdot (a_k \oplus j_k) = i_k \cdot 0 = 0$ so for all i ,

$$\begin{array}{c} \pi \\ | \\ \triangle \\ f_a(i) \oplus (i \cdot j) \end{array} = \begin{array}{c} \pi \\ | \\ \triangle \\ 0 \end{array} = \boxed{}$$

Summing them all up, we still have

$$\frac{1}{2^n} \sum_{i \in \{0,1\}^n} \boxed{} = \boxed{}$$

On the other hand, we look at what happens when

$$\begin{array}{c} | \\ \triangle \\ j \end{array} \neq \begin{array}{c} | \\ \triangle \\ a \end{array}$$

So it must be the case that

$$\begin{array}{c} | \\ \triangle \\ a \oplus j \end{array} = \begin{array}{c} | \\ \triangle \\ b_1 \end{array} \dots \begin{array}{c} | \\ \triangle \\ 1 \end{array} \dots \begin{array}{c} | \\ \triangle \\ b_n \end{array}$$

where

$$\begin{array}{c} | \\ \triangle \\ b_k \end{array} = \begin{array}{c} | \\ \triangle \\ 1 \end{array}$$

for some bit b_k at index k . So we can find two inputs $i, i' \in \{0,1\}^n$ where

$$\begin{array}{c} | \\ \triangle \\ i \end{array} = \begin{array}{c} | \\ \triangle \\ b_1 \end{array} \dots \begin{array}{c} | \\ \triangle \\ 1 \end{array} \dots \begin{array}{c} | \\ \triangle \\ b_n \end{array}$$

$$\begin{array}{c} | \\ \triangle \\ i' \end{array} = \begin{array}{c} | \\ \triangle \\ b_1 \end{array} \dots \begin{array}{c} | \\ \triangle \\ 0 \end{array} \dots \begin{array}{c} | \\ \triangle \\ b_n \end{array}$$

so that

$$\begin{array}{c} \pi \\ | \\ \triangle \\ i \cdot (a \oplus j) \end{array} + \begin{array}{c} \pi \\ | \\ \triangle \\ i' \cdot (a \oplus j) \end{array} = 0$$

We can then find all possible inputs pairs of $i, i' \in \{0, 1\}^n$ corresponding to the 2^{n-1} choices for $b_1, \dots, b_n$ with $b_k = 0$ and the 2^{n-1} choices for $b_1, \dots, b_n$ with $b_k = 1$, leading to all inputs having a corresponding pair. Then when

$$\downarrow_j \neq \downarrow_a,$$

we have, as required,

$$\frac{1}{2^n} \sum_{i \in \{0,1\}^n} \begin{array}{c} \circ \pi \\ | \\ \triangle f_a(i) \oplus (i \cdot j) \end{array} = 0$$

□

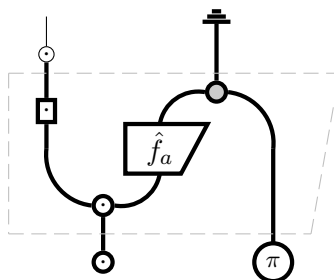
Now, we can use the Lemma 1.12 above to show

$$\frac{1}{2^n} \sum_{i,j \in \{0,1\}^n} \begin{array}{c} \circ \pi \\ | \\ \triangle f_a(i) \oplus (i \cdot j) \end{array} \downarrow_j = \downarrow_a$$

We then just have to measure the result to obtain our string a . We have our quantum algorithm as desired.

Quantum Algorithm

Quantum Algorithm. Perform:



Then we will always find string a . The algorithm as shown is named the *Bernstein–Vazirani algorithm*.

In [4], the algorithm is refined to use n bits instead of $n + 1$, and implemented on an ensemble quantum computer.

Extension

A simple extension of the *Bernstein–Vazirani algorithm* is shown in [7] and extends the oracle function to having k secret strings.

Given. A function

$$f : \{0, 1\}^n \rightarrow \{0, 1\}$$

with the promise that, for k n -bit strings $a_1, a_2, \dots, a_k \in \{0, 1\}^n$, where $1 \leq k \leq 2^n$,

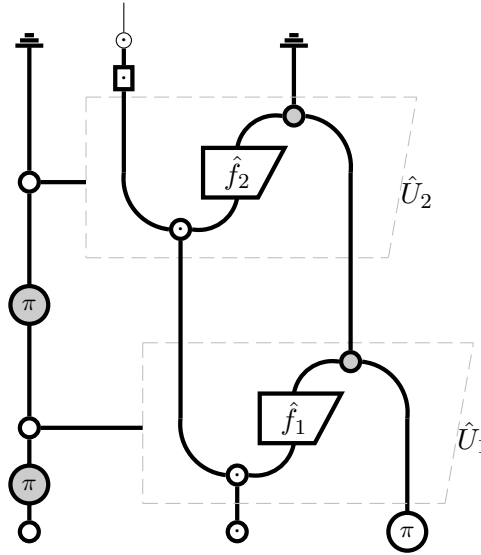
$$f_i(x) = a_i \cdot x \equiv \left(\sum_{i=1}^n a_i x_i \right) \pmod{2},$$

where a_i, x_i are the i -th bits of a, x respectively, and $f(x) = f_i(x)$ with equal probability of $\frac{1}{k}$ for each $i \in \{1, \dots, k\}$.

Problem. What are the exact strings $a_1, \dots, a_k$?

This paper will show the diagrammatic reasoning for $k = 2$.

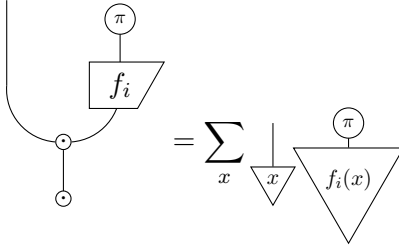
It is assumed in [7] that each unitary U_i corresponds to the function f_i (and thus string a_i). Using **Section 12.1.3.2** from [2], we can construct controlled unitaries, pictured as follows.



Since the input to the control qubit is

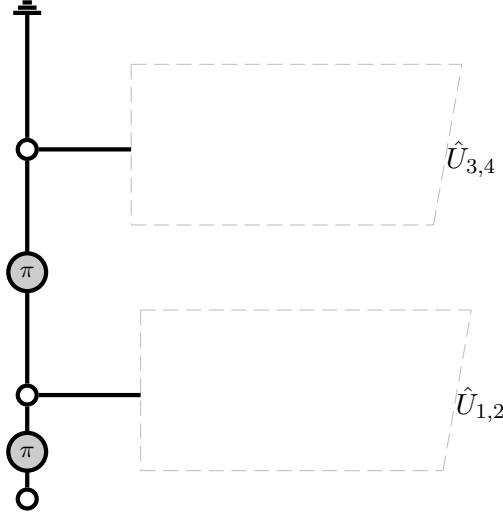
$$|0\rangle = \text{double} \left(\frac{|0\rangle}{\sqrt{2}} + \frac{|1\rangle}{\sqrt{2}} \right) = \frac{1}{2} \left(|0\rangle + |pi\rangle \right),$$

both unitaries will be "fired" with equal probability of $\frac{1}{2}$. The resulting measurement will then give each string a_1, a_2 with probability $\frac{1}{2}$. The analysis is similar to before, where unitary U_i firing for $i \in \{1, 2\}$ will result in



and the subsequent n -bit Hadamard will allow the string a_i to be easily measured.

As shown in [7], there is no classical algorithm that can obtain even a single bit of any key, making the quantum algorithm superior. [7] goes on to generalize the algorithm for $k = 2^r$, which can be easily obtained. Below, $k = 4$ is shown, using 'controlled-controlled- $\hat{U}$ ' gates as detailed in **Section 12.1.3.2** of [2]. Within the dotted lines contain the existing controlled- $\hat{U}$ gates for $k = 2$ similar to before, and it is straightforward to apply the same principle of turning "on" and "off" entire sets of unitaries.



When $k \neq 2^r$, the uniform superposition state requires the Quantum Fourier Transform and is outside the scope of this paper.

Conclusion

The *Bernstein–Vazirani algorithm* has been shown as presented in [1][8][4], as well as an extension found in [7]. It has been proven diagrammatically, with techniques found in lectures and [2]. It is vastly superior to the classical algorithms, improving the number of queries from $O(n)$ to $O(1)$ in the single string case, and obtaining each key with certainty as opposed to not even a single bit with certainty, in the multi string case.

References

- [1] Ethan Bernstein and Umesh Vazirani. Quantum complexity theory. *SIAM Journal on Computing*, 26(5):1411–1473, 1997.
- [2] Bob Coecke and Aleks Kissinger. *Picturing Quantum Processes*. Cambridge University Press, 2017.
- [3] D Deutsch and R Jozsa. Rapid solution of problems by quantum computation. Technical report, GBR, 1992.
- [4] Jiangfeng Du, Mingjun Shi, Xianyi Zhou, Yangmei Fan, BangJiao Ye, Rongdian Han, and Jihui Wu. Implementation of a quantum algorithm to solve the bernstein-vazirani parity problem without entanglement on an ensemble quantum computer. *Physical Review A*, 64(4), September 2001.
- [5] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '96, page 212–219, New York, NY, USA, 1996. Association for Computing Machinery.
- [6] Aleks Kissinger and John van de Wetering. Universal mbqc with generalised parity-phase interactions and pauli measurements. *Quantum*, 3:134, April 2019.
- [7] Alok Shukla and Prakash Vedula. A generalization of bernstein–vazirani algorithm with multiple secret keys and a probabilistic oracle. *Quantum Information Processing*, 22(6), June 2023.
- [8] Barbara M. Terhal and John A. Smolin. Single quantum querying of a database. *Physical Review A*, 58(3):1822–1826, September 1998.